

```
=====
=====
P2P SKILL EXCHANGE - IMPLEMENTATION ORDER & QUICK START GUIDE
=====
=====
```

BEFORE YOU START:

1. Make sure you have Node.js installed (v14+)
2. MongoDB account (Atlas or local instance)
3. Git initialized with GitHub/GitLab
4. Code editor (VS Code recommended)
5. Postman or Insomnia for API testing

```
=====
=====
PHASE 0: INITIAL SETUP (Day 1-2)
=====
=====
```

BACKEND SETUP:

1. Initialize Node project

```
$ mkdir backend && cd backend
$ npm init -y
```
2. Install dependencies from package.json

```
$ npm install
```
3. Create directory structure

```
$ mkdir config models middleware services routes utils
```
4. Create .env file from .env.example

```
$ cp .env.example .env
# Fill in MongoDB URI, JWT secret, etc.
```
5. Test server startup

```
$ npm run dev
# Should start on http://localhost:5000
```

FRONTEND SETUP:

1. Create React app

```
$ npx create-react-app frontend
```

2. Install additional dependencies from package.json

```
$ cd frontend
```

```
$ npm install
```

3. Test frontend startup

```
$ npm start
```

```
# Should open http://localhost:3000
```

4. Update proxy in package.json for API calls

```
"proxy": "http://localhost:5000"
```

```
=====
```

```
=====
```

PHASE 1: FOUNDATION - Backend Configuration (Day 3)

```
=====
```

```
=====
```

PRIORITY: HIGH | EFFORT: 4 hours

Files to Create:

```
|— backend/server.js
|— backend/config/config.js
|— backend/config/database.js
```

WHAT TO ADD:

1. server.js

```
// Import Express, dotenv, database
// Load .env variables
// Create Express app
// Setup middleware (CORS, bodyParser, helmet)
// Connect to MongoDB
// Register routes (import all route files)
// Setup error handling middleware
// Start server on PORT
```

2. config/config.js

```
// Export environment-specific settings
// Database, JWT, Bcrypt, Email, Credit constants
// Based on NODE_ENV (development/production)
```

3. config/database.js

```
// Connect to MongoDB
// Handle connection events
```

```
// Export connect() function
```

TEST:

```
$ npm run dev
```

- ✓ Server starts without errors
- ✓ MongoDB connection logs appear
- ✓ No errors in console

```
=====
```

```
=====
```

PHASE 2: DATA MODELS - Database Schemas (Day 4-5)

```
=====
```

```
=====
```

PRIORITY: HIGH | EFFORT: 6 hours

Files to Create:

- └─ backend/models/User.js
- └─ backend/models/Skill.js
- └─ backend/models/Session.js
- └─ backend/models/Wallet.js
- └─ backend/models/Review.js
- └─ backend/models/Notification.js
- └─ backend/models/Report.js

WHAT TO ADD in each:

1. User.js

```
// Schema fields (email, phone, password, profile, skills, etc.)  
// Methods: comparePassword(), generateAuthToken()  
// Pre-save hook: hash password if modified  
// Indexes: email, phone (unique)  
// Export User model
```

2. Skill.js

```
// Schema: skillName, category, description, level, credits  
// Fields: userId, tags, availability, isActive  
// Indexes: skillName, category, userId  
// Export Skill model
```

3. Session.js

```
// Schema: mentorId, learnerId, skillId, sessionDate  
// Status field: REQUESTED, ACCEPTED, COMPLETED, CANCELLED  
// Credits: creditsExchanged
```

```
// Indexes: mentorId, learnerId, sessionId
// Export Session model
```

4. Wallet.js

```
// Schema: userId, currentBalance, totalEarned, totalSpent
// Transactions array: type, amount, sessionId, timestamp
// Methods: addCredits(), deductCredits(), checkBalance()
// Index: userId
// Export Wallet model
```

5. Review.js

```
// Schema: sessionId, mentorId, learnerId
// Rating (1-5), reviewText
// reportedAsInappropriate flag
// Index: mentorId, learnerId, sessionId
// Export Review model
```

6. Notification.js

```
// Schema: userId, type, title, message
// isRead, channels (EMAIL, IN_APP, PUSH)
// relatedId (for linking to sessions/skills)
// Index: userId, isRead
// Export Notification model
```

7. Report.js

```
// Schema: reporterId, reportedUserId, reportedItemId
// reportType: INAPPROPRIATE_SKILL, DISPUTE, ABUSE, etc.
// status: PENDING, IN_REVIEW, RESOLVED
// adminNotes, action taken
// Index: status, reportedUserId
// Export Report model
```

TEST:

```
$ node -e "require('./models/User'); console.log('User model loaded')"
```

- ✓ All models load without errors
- ✓ Database connection established
- ✓ No schema validation errors

```
=====
=====
PHASE 3: MIDDLEWARE SETUP (Day 5-6)
=====
=====
```

PRIORITY: HIGH | EFFORT: 6 hours

Files to Create:

- |— backend/middleware/authMiddleware.js
- |— backend/middleware/authorizationMiddleware.js
- |— backend/middleware/validationMiddleware.js
- |— backend/middleware/errorHandler.js

WHAT TO ADD:

1. authMiddleware.js

- // Check for Authorization header
- // Verify JWT token
- // Extract userId and fetch user
- // Attach req.user = user
- // Handle errors (invalid token, expired token)
- // Export verifyAuth and optionalAuth

2. authorizationMiddleware.js

- // Factory function accepting array of allowed roles
- // Check if req.user.role is in allowed roles
- // 403 Forbidden if not authorized
- // Export: adminOnly(), mentorOnly(), learnerOnly(), ownerOnly()

3. validationMiddleware.js

- // Create Joi schemas for:
- // - registerSchema
- // - loginSchema
- // - profileUpdateSchema
- // - skillCreateSchema
- // - sessionRequestSchema
- // - reviewSchema
- // Create middleware factory that validates request
- // Return validation errors if invalid
- // Export all schemas and middleware

4. errorHandler.js

- // Create custom error classes
- // Create error handling middleware
- // Log errors with context
- // Return formatted error responses
- // Export errors and handler

TEST:

\$ npm run dev

- ✓ Middleware loads without errors
- ✓ Can make test requests and see middleware in action

```
=====
=====
PHASE 4: AUTHENTICATION LAYER (Day 6-7)
=====
=====
```

PRIORITY: CRITICAL | EFFORT: 8 hours

Files to Create:

```
|— backend/services/authService.js
|— backend/routes/authRoutes.js
|— frontend/src/redux/slices/authSlice.js
```

WHAT TO ADD:

BACKEND:

1. services/authService.js

```
// register(email, phone, password, firstName, lastName)
// - Check user exists
// - Hash password with bcrypt
// - Create User
// - Initialize Wallet
// - Send verification email
// - Return user + token
// login(email/phone, password)
// - Find user
// - Compare password
// - Generate token
// generateToken(userId)
// verifyToken(token)
// resetPassword(email, newPassword)
```
2. routes/authRoutes.js

```
// POST /register - Call authService.register()
// POST /login - Call authService.login()
// POST /logout - Clear token
// POST /forgot-password - Call resetPassword flow
// POST /reset-password - Update password
// Apply validation middleware
```

// Apply error handling

FRONTEND:

1. redux/slices/authSlice.js

// Create async thunks:
// - registerUser
// - loginUser
// - logoutUser
// - verifyToken
// State: user, token, isAuthenticated, loading, error
// Reducers: setUser, clearAuth, setLoading
// Selectors: selectUser, selectToken, selectIsAuth
// Save token to localStorage on login

TEST with Postman:

- ✓ POST /auth/register → Returns user + token
- ✓ POST /auth/login → Returns user + token
- ✓ POST /auth/logout → Clears session
- ✓ POST /auth/reset-password → Updates password

=====

=====

PHASE 5: USER PROFILE MANAGEMENT (Day 8-9)

=====

=====

PRIORITY: HIGH | EFFORT: 6 hours

Files to Create:

- └ backend/services/userService.js
- └ backend/routes/userRoutes.js
- └ frontend/src/redux/slices/userSlice.js
- └ frontend/src/services/userAPI.js

WHAT TO ADD:

BACKEND:

1. services/userService.js

// getUserProfile(userId)
// updateUserProfile(userId, updateData)
// uploadProfilePicture(userId, file)
// addSkillToTeach(userId, skillData)

```
// removeSkill(userId, skillId, type)
// setAvailability(userId, availabilityData)
```

2. routes/userRoutes.js

```
// GET /profile → userService.getUserProfile()
// PUT /profile → userService.updateUserProfile()
// POST /profile/picture → Upload file
// POST /skills/teach → Add skill
// DELETE /skills/teach/:skillId → Remove skill
// PUT /availability → Set schedule
// Apply authMiddleware to all routes
```

FRONTEND:

1. redux/slices/userSlice.js

```
// Async thunks for all user operations
// State management for profile, skills, availability
```

2. services/userAPI.js

```
// getUserProfile()
// updateUserProfile()
// uploadProfilePicture()
// Skill management API calls
```

TEST:

- ✓ Can fetch user profile
- ✓ Can update profile fields
- ✓ Can upload profile picture
- ✓ Can add/remove skills

=====

=====

PHASE 6: SKILL MANAGEMENT (Day 10-11)

=====

=====

PRIORITY: HIGH | EFFORT: 8 hours

Files to Create:

- └ backend/services/skillService.js
- └ backend/routes/skillRoutes.js
- └ frontend/src/redux/slices/skillSlice.js
- └ frontend/src/services/skillAPI.js

WHAT TO ADD:

BACKEND:

1. services/skillService.js
 - // createSkill(userId, skillData)
 - // updateSkill(skillId, userId, updateData)
 - // deleteSkill(skillId, userId)
 - // getSkillById(skillId)
 - // getAllSkills(filters, page, limit)
 - // searchSkills(query, filters)
 - // getRecommendedMentors(learnerId, skillName)
2. routes/skillRoutes.js
 - // GET /skills → getAllSkills
 - // GET /skills/search → searchSkills
 - // GET /skills/trending → getTrendingSkills
 - // GET /skills/recommendations → getRecommendedMentors
 - // GET /skills/:skillId → getSkillById
 - // POST /skills → createSkill (auth required)
 - // PUT /skills/:skillId → updateSkill
 - // DELETE /skills/:skillId → deleteSkill

FRONTEND:

1. redux/slices/skillSlice.js
 - // Async thunks for skill operations
 - // State: skills, filters, sorting, page
 - // Reducers: setFilters, setSorting, setPage
2. services/skillAPI.js
 - // API calls to all skill endpoints

TEST:

- ✓ Can search skills
- ✓ Can filter by category, level, rating
- ✓ Can create/update/delete skills
- ✓ Can get mentor recommendations

=====

=====

PHASE 7: SESSION MANAGEMENT (Day 12-13)

=====

=====

PRIORITY: CRITICAL | EFFORT: 10 hours

Files to Create:

- |— backend/services/sessionService.js
- |— backend/services/walletService.js
- |— backend/routes/sessionRoutes.js
- |— backend/routes/walletRoutes.js
- |— frontend/src/redux/slices/sessionSlice.js
- |— frontend/src/redux/slices/walletSlice.js
- |— frontend/src/services/sessionAPI.js
- |— frontend/src/services/walletAPI.js

WHAT TO ADD:

BACKEND:

1. services/sessionService.js

```
// requestSession(learnerId, mentorId, skillId, sessionId)
// - Check learner balance
// - Check mentor availability
// - Check double booking
// - Create Session (REQUESTED status)
// - Notify mentor
// acceptSession(sessionId, mentorId)
// rejectSession(sessionId, mentorId)
// cancelSession(sessionId, userId)
// - Refund credits if needed
// rescheduleSession(sessionId, userId, newDateTime)
// completeSession(sessionId)
// - Call walletService.transferCredits()
// checkDoubleBooking(mentorId, dateTime)
```

2. services/walletService.js

```
// initializeWallet(userId)
// addCredits(userId, amount, sessionId)
// deductCredits(userId, amount, sessionId)
// checkSufficientBalance(userId, amount)
// transferCredits(fromUserId, toUserId, amount)
// - ATOMIC: both succeed or both fail
// getWalletBalance(userId)
// getTransactionHistory(userId, filters)
```

3. routes/sessionRoutes.js

```
// POST /request → requestSession
// GET /requests/incoming → getIncomingRequests (mentor)
// GET /requests/outgoing → getOutgoingRequests (learner)
// PUT /:sessionId/accept → acceptSession
// PUT /:sessionId/reject → rejectSession
// PUT /:sessionId/cancel → cancelSession
// PUT /:sessionId/reschedule → rescheduleSession
// PUT /:sessionId/complete → completeSession
// GET /upcoming → getUpcomingSessions
```

4. routes/walletRoutes.js

```
// GET /balance → getWalletBalance
// GET /transactions → getTransactionHistory
// GET /stats → getWalletStats
// POST /verify-balance/:amount → checkBalance
```

FRONTEND:

1. redux/slices/sessionSlice.js & walletSlice.js
2. services/sessionAPI.js & walletAPI.js

TEST:

- ✓ Can request session (check credit deduction)
- ✓ Can accept/reject/cancel sessions
- ✓ Credits transfer correctly on completion
- ✓ Wallet balance updates
- ✓ Double booking prevented
- ✓ Transaction history recorded

=====

=====

PHASE 8: REVIEWS & RATINGS (Day 14)

=====

=====

PRIORITY: MEDIUM | EFFORT: 4 hours

Files to Create:

```
├─ backend/services/reviewService.js
├─ backend/routes/reviewRoutes.js
├─ frontend/src/services/reviewAPI.js
```

WHAT TO ADD:

BACKEND:

1. services/reviewService.js

```
// submitReview(sessionId, learnerId, rating, reviewText)
// - Verify session completed
// - Prevent duplicate reviews
// - Create Review
// - Recalculate mentor average rating
// getReviewsForMentor(mentorId, page, limit)
// getMentorAverageRating(mentorId)
// updateReview(reviewId, learnerId, updateData)
// deleteReview(reviewId, learnerId)
// reportReview(reviewId, reason)
```

2. routes/reviewRoutes.js

```
// POST / → submitReview
// GET /mentor/:mentorId → getReviewsForMentor (public)
// PUT /:reviewId → updateReview (auth required)
// DELETE /:reviewId → deleteReview (auth required)
// GET /mentors/:mentorId/rating → getMentorRating (public)
// POST /:reviewId/report → reportReview
```

TEST:

- ✓ Can submit review after session
- ✓ Cannot submit duplicate review
- ✓ Mentor average rating updates
- ✓ Can view mentor reviews publicly
- ✓ Can report inappropriate reviews

=====

=====

PHASE 9: NOTIFICATIONS & EMAIL (Day 15)

=====

=====

PRIORITY: MEDIUM | EFFORT: 6 hours

Files to Create:

- └─ backend/services/notificationService.js
- └─ backend/services/emailService.js
- └─ backend/routes/notificationRoutes.js

WHAT TO ADD:

BACKEND:

1. services/emailService.js
// Initialize nodemailer transport
// sendVerificationEmail()
// sendPasswordResetEmail()
// sendSessionRequestEmail()
// sendWelcomeEmail()
// sendSessionReminderEmail()
// sendTransactionEmail()
2. services/notificationService.js
// sendNotification(userId, type, title, message, channels)
// sendSessionRequestNotification()
// sendRequestAcceptedNotification()
// sendSessionReminderNotification()
// sendCreditUpdateNotification()
3. routes/notificationRoutes.js
// GET / → getUserNotifications
// GET /unread-count → getUnreadCount
// PUT /:notificationId/read → markAsRead
// DELETE /:notificationId → deleteNotification

TEST:

- ✓ Emails sent on registration
- ✓ Notifications sent on session request
- ✓ Notifications sent on acceptance
- ✓ In-app notifications display

=====

=====

PHASE 10: ADMIN PANEL (Day 16-17)

=====

=====

PRIORITY: MEDIUM | EFFORT: 8 hours

Files to Create:

- └─ backend/services/adminService.js
- └─ backend/routes/adminRoutes.js

WHAT TO ADD:

BACKEND:

```
1. services/adminService.js
  // getUserList(filters, page)
  // suspendUser(userId, reason)
  // activateUser(userId)
  // removeSkillListing(skillId, reason)
  // getReportedIssues(status, page)
  // resolveReport(reportId, adminId, action, notes)
  // getSystemAnalytics()
  // getCreditTransactionReport()
  // getUserAnalytics(userId)

2. routes/adminRoutes.js
  // GET /users → getUserList
  // GET /users/:userId → getUserDetails
  // PUT /users/:userId/suspend → suspendUser
  // PUT /users/:userId/activate → activateUser
  // DELETE /skills/:skillId → removeSkillListing
  // GET /reports → getReportedIssues
  // PUT /reports/:reportId/resolve → resolveReport
  // GET /analytics → getSystemAnalytics
  // Apply adminOnly middleware to all
```

TEST:

- ✓ Admin can view all users
- ✓ Admin can suspend/activate users
- ✓ Admin can remove skills
- ✓ Admin can resolve reports
- ✓ Admin can view analytics

=====

=====

PHASE 11: FRONTEND PAGES & COMPONENTS (Day 18-25)

=====

=====

Create all frontend pages and components following the component structure.

Priority order:

1. Common Components (Header, Footer, Navigation)
2. Auth Pages (Login, Register)
3. Dashboard Page
4. Profile Page

5. Skill Pages
6. Session Pages
7. Wallet Page
8. Admin Pages
9. Review Pages
10. Notification Center

=====

=====

PHASE 12: INTEGRATION & TESTING (Day 26-27)

=====

=====

1. End-to-end testing of all features
2. Error handling and edge cases
3. Performance optimization
4. Security review
5. User acceptance testing

=====

=====

QUICK COMMAND REFERENCE

=====

=====

Backend Development:

\$ npm run dev # Start backend with nodemon

Testing APIs:

- Use Postman or Insomnia
- Test each endpoint before moving to frontend

Frontend Development:

\$ npm start # Start React dev server

\$ npm build # Build for production

Database:

- Check MongoDB for data
- Verify collections are created

Error Debugging:

- Backend: Check console for error logs
- Frontend: Use Redux DevTools in Chrome
- Network: Check browser Network tab and console

File Watching:

- Backend automatically restarts with nodemon
- Frontend automatically reloads with create-react-app

```
=====
=====
DEPENDENCIES QUICK CHECK
=====
=====
```

Backend Essential:

- ✓ express
- ✓ mongoose
- ✓ bcryptjs
- ✓ jsonwebtoken
- ✓ dotenv
- ✓ joi (validation)
- ✓ nodemailer (email)
- ✓ cors
- ✓ helmet

Frontend Essential:

- ✓ react
- ✓ react-dom
- ✓ react-router-dom
- ✓ redux
- ✓ react-redux
- ✓ @reduxjs/toolkit
- ✓ axios
- ✓ formik & yup (forms)
- ✓ react-icons

```
=====
=====
```

ESTIMATED TIMELINE

With this structure and implementation order:

- Week 1: Foundation + Auth (40 hours)
- Week 2: Users + Skills (40 hours)
- Week 3: Sessions + Wallet (40 hours)
- Week 4: Reviews + Notifications (40 hours)
- Week 5: Admin + Frontend Integration (40 hours)

Total: ~200 development hours
With breaks and reviews: ~5 weeks for 1 developer
~1-2 weeks for 6 developers (parallel work)

=====

NOW YOU'RE READY TO START CODING!

Begin with Phase 0: Initial Setup, then follow the phases in order.
Each phase builds on the previous one.
Test thoroughly at each phase before moving to the next.

Good luck! 🚀

=====